

SQL Views – Research & Implementation Task

1. Types of Views in SQL Server

1.1 Standard View

- **What is it?**

A standard view is a virtual table representing the result of a SELECT query. It does not store data physically but provides a way to simplify complex queries or restrict access to sensitive data.

- **Key Differences:**

- Does not store data; retrieves data dynamically from base tables.
- Easier to maintain than rewriting queries multiple times.
- Supports DML operations if certain conditions are met.

- **Real-life Use Cases:**

- **Banking:** Summarizing account balances per customer without exposing individual transactions.
- **E-commerce:** Showing product inventory along with supplier info in one view.
- **University System:** Displaying students along with their course enrollments.

- **Limitations and Performance Considerations:**

- Performance may be slower for very large datasets since queries are executed at runtime.
- Cannot include certain constructs like TOP without ORDER BY in some contexts.

1.2 Indexed View

- **What is it?**

An indexed view (materialized view) stores the result of the view physically in the database. This improves performance for queries that involve complex aggregations or joins.

- **Key Differences:**

- Data is physically stored and maintained automatically.
- Provides faster query performance but requires storage space.
- Only allowed on deterministic functions and schemas with certain restrictions.

- **Real-life Use Cases:**

- **Banking:** Pre-computed summaries of daily transaction totals for reporting.
- **E-commerce:** Frequently accessed sales totals per category.
- **University System:** Cached GPA calculations for students.

- **Limitations and Performance Considerations:**

- Increases storage usage.
- Adds overhead on insert/update/delete operations on underlying tables.
- Cannot include non-deterministic functions like GETDATE().

1.3 Partitioned View (Union View)

- **What is it?**

A partitioned view combines multiple tables using UNION ALL, often across different filegroups or databases. It's useful for horizontal partitioning of data.

- **Key Differences:**

- Physically separates data across tables but presents it as a single logical table.
- Often used for large datasets to improve manageability and query performance.
- **Real-life Use Cases:**
 - **Banking:** Splitting customer transaction history by year or region, but querying as a single dataset.
 - **E-commerce:** Historical sales data partitioned by year.
 - **University System:** Student records partitioned by graduation year.
- **Limitations and Performance Considerations:**
 - Requires consistent schema across partitions.
 - Queries with joins can be slower if partitions are distributed across servers.
 - Complexity in maintaining partitions.

2. Can We Use DML (INSERT, UPDATE, DELETE) on Views?

- **Which types allow DML operations:**
 - **Standard Views:** Allow DML if they reference a single base table, do not contain aggregates, DISTINCT, GROUP BY, UNION, or computed columns.
 - **Indexed Views:** DML can be performed but with restrictions on indexed columns and schema binding.
 - **Partitioned Views:** DML is possible but must target the correct underlying table (check constraints help direct operations).
- **Restrictions/Limitations:**
 - Cannot insert into a view that has joins (unless INSTEAD OF triggers are defined).
 - Updates may be restricted to columns from a single table.
 - Cannot delete rows if the view includes aggregates or DISTINCT.
- **Real-life Example:**
 - **HR System:** Updating employee contact info through a view that joins Employee and Department tables, so users cannot access salary data.

Example in SQL Server:

sql

 Copy code

```
-- Standard view example
CREATE VIEW vw_CustomerAccounts AS
SELECT CustomerID, Name, AccountNumber, Balance
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID;

-- Update through the view (allowed since it's single table for each row)
UPDATE vw_CustomerAccounts
SET Balance = Balance + 100
WHERE CustomerID = 1;
```

3. How Can Views Simplify Complex Queries?

- **Explanation:**

Views act as reusable query objects. Instead of repeating a long JOIN query every time, a view stores the logic, allowing users to query it as a simple table. This reduces errors and simplifies maintenance.

- **Banking Example:**

Frequent query: “Get account summary and latest transaction for call center agents.”

sql

 Copy code

```
-- Create view
CREATE VIEW vw_AccountTransactionSummary AS
SELECT a.AccountNumber, a.Balance, t.TransactionDate, t.Amount, c.Name AS CustomerName
FROM Accounts a
JOIN Customers c ON a.CustomerID = c.CustomerID
JOIN Transactions t ON a.AccountNumber = t.AccountNumber
WHERE t.TransactionDate = (SELECT MAX(TransactionDate)
                           FROM Transactions t2
                           WHERE t2.AccountNumber = a.AccountNumber);

-- Using the view
SELECT * FROM vw_AccountTransactionSummary
WHERE CustomerName LIKE 'John%';
```

Benefits:

- No need to rewrite the JOIN every time.
- Simplifies access for end-users.
- Can encapsulate security by limiting access to sensitive columns.